Python for Machine Learning

Last updated: November 10, 2023

Written by: Nikhil Bhargav
(https://www.baeldung.com/cs/author/nikhilbhargav)

Machine Learning (https://www.baeldung.com/cs/category/ai/ml)

Programming (https://www.baeldung.com/cs/category/core-concepts/programming)

Project Management (https://www.baeldung.com/cs/tag/project-management)

1. Introduction

In this tutorial, we'll see why Python (/cs/max-int-java-c-python) is the programming language of choice for machine learning (/cs/analytical-inductive-learning-prior-knowledge).

2. Machine Learning and Programming

We can implement machine-learning algorithms in any high-level (Python, R, Matlab), low-level programming language (C, C++), or a combination thereof.

However, we usually choose a high-level language to write more understandable and maintainable code faster and with high accuracy. For this, we use a combination of languages for modern software development in machine learning.

Going with a high-level language provides a clean, compact, and rich interface to code any machine-learning algorithm. Most of the details are taken care of by the language or the library so that we can focus on the ideas and the logic.

Let's check out what makes Python stand out.

3. Python Programming Language

Python is designed for quality, productivity, portability, and integration.

Python code is highly readable, simple, and scalable because of its clear syntax and coherent design. We can learn it very fast. Its programs look like algorithms, which allows us to focus on ideas, not implementational details incidental to the algorithmic logic.

Python increases development speed. We find it easy to write programs fast in Python. This is so because the interpreter handles most details, such as type declarations, storage, memory management, interrupt handling, and build procedures. This makes Python flexible and agile.

It's open-source and accessible and has a large community base. Its standard library contains diverse packages and modules for different domains.

(/cs/) (https://www.baeldung.com/cs/)
(/bael-search)

Python is also portable (/cs/software-quality). We can write *Python* code on Windows and run it on Mac and Linux without modification. This is because the core part of *Python* language and its libraries are platform-neutral. For edge cases of platform incompatibility, Python provides tools to build dependencies.

Python is highly integrative. We can mix up Python code easily with other system components at any level. For example, we can call a fast *C/C++* from within Python.

3.1. Example

Let's consider a Python function for linear search (/cs/linear-search-faster):

```
def linear_search(input_list, search_item):
    for item in input_list:
        if item == search_item:
            return True
    return False
```

It reads like an algorithm since it doesn't deal with details such as data types and list indices.

In contrast, the implementation of the same algorithm in *C++* deals with more details since we need to take care of memory management, types, and array indices:

```
bool linear_search(int *input_list, int size, int search_item){
    for (int i=0; i < size; i++){
        if (input_list[i] == search_item){
             return true;
         }
     }
     return false;
}
```

We'll have to write another function for other data types to do the linear search. This isn't the case with Python because of its dynamic typing (/cs/statically-vs-dynamically-typed-languages).

4. Python for Machine Learning

Python has many libraries for every domain.

We use *numpy* (/cs/svm-multiclass-classification) for doing scientific computing using an architecture-efficient representation of data. The *pandas (/cs/naive-bayes-classification)* library is for data analysis and manipulation of structured and unstructured data. Going further, we use *matplotlib* (/cs/reinforcement-learning-neural-network) to plot all our figures and charts. These libraries don't deal with machine learning per see but support us in building and testing ML models.

4.1. Key Machine-Learning Frameworks

Let's examine four key machine-learning Python frameworks: *scikit-learn* (https://scikit-learn.org/stable/), *Keras* (https://keras.io/), *TensorFlow* (https://www.tensorflow.org/), and *PyTorch* (https://pytorch.org/).

scikit-learn or *sklearn* is the most used Python library for machine learning. It is robust, scalable and expanding. We get many well-tested and documented tools for classification, regression (/cs/normalization-vs-standardization), clustering (/cs/dbscan-algorithm), and dimensionality reduction tasks (/cs/feature-selection-reduction-for-text-classification).

TensorFlow is an open-source and complete platform we can use for multiple machine-learning tasks. Google developed it and made it public in 2015. It provides a rich abstraction for building and training custom models in deep learning. TensorFlow has a vibrant ecosystem of community resources, libraries, and tools for building and deploying (/cs/continuous-integration-deployment-delivery) machine learning apps that can scale fast.

On the other hand, *Keras* is a high-level neural network library that is a wrapper (/java-wrapper-classes) over *TensorFlow*. *Keras* is more user-friendly because it hides all the low-level details of *Tensorflow* from users. So, we can use *Keras* to define our model since it's easier to use. If we need a specific *TensorFlow* functionality that *Keras* doesn't offer, we can directly use *TensorFlow*.

PyTorch is a relatively newer deep-learning (/cs/end-to-end-deep-learning) framework that is based on *Torch*. Facebook's AI research group developed it open-sourced in 2017. Compared to *TensorFlow*, we find it simpler and easier

to use. It also gives us more flexibility and better memory management. Additionally, it uses dynamic computational graphs to train models. Further, *PyTorch* is faster and makes our code more manageable and understandable.

(/cs/) (https://www.baeldung.com/cs/)
(/bael-search)

5. Alternatives

This section explores alternative programming languages we can use for machine learning.

5.1. C/C++

C/C++ has a faster run-time than its peers, so it is suitable for implementing those machine learning problems that mandate low latency without compromising accuracy. C++ also has rich library support for machine learning.

On the flip side, its drawback is writing less code for the same problem than Python. This is because we must specify most implementation details directly in our code.

We can appreciate these ideas by looking at a simple linear regression model in C++, available on GitHub (https://github.com/Baeldung/posts-resources/tree/main/cs-articles/python-and-machine-learning). It's lengthy and very low-level.

5.2. Matlab

Matlab provides a numerical computing-based environment that supports various languages. It's ideal for implementing machine learning algorithms that are mathematically heavy (a lot of statistics and calculus use).

Matlab provides the fastest execution time, the best-integrated development environment, rich libraries, and other language integrations. However, it has a steep learning rate since it is not compact and flexible. Further, it is licensed software and thus has limited adoption in the industry.

(/cs/) (https://www.baeldung.com/cs/)

Moreover, we need help seeing the underlying Matlab code of key algorithms since it provides them as apps or modules. Thus, debugging Matlab code takes a lot of work.

(/bael-search)

Adding to the problems, **Matlab code isn't portable.**

5.3. R

R is an open-source programming language for statistical programming and visualisation. It's derived from Lisp and, thus, is an interpreted language.

R is suitable for prototyping algorithms in research but not so much for industrial adoption. Moreover, we find its documentation complex; thus, it has a steep learning curve. Furthermore, R needs more consistency in implementing machine learning algorithms since its libraries use different naming conventions and design styles.

5.4. Node.js

We primarily use Node.js for server-side applications. The developer community has started using Node.js for machine learning in the last few years.

Most major machine learning libraries like OpenCV, TensorFlow, and NumPy have Node.js implementations. These libraries provide easy-to-use machine learning models so that we can train and deploy our solutions directly within a Node.js environment.

6. Comparison

In this section, we compare all candidate programming languages for machine learning:

Criteria	Python	C/C++	MATLAB	R	Nodejs
Usability	Simple	Complex	Complex	Complex	Simple
Licencing	Open-source	Open-source	Commercial	Open-source	Open-source
Abstraction	High-level	Low-level	Low-level	High-level	High-level
Library support	Very high	Limited	Medium	Limited	High
User base	highest	Low	Very low	Low	Medium
Applicability	All apps	Real-time apps	Scientific apps	Visualizing apps	Web apps

This table shows that Python is the most suitable language to code any machine-learning solution.

7. Conclusion

This article studied Python as the core programming language for machine learning problems.

To conclude, we find Python outscores other languages for implementing machine learning algorithms because of its simplicity, flexibility, rich library support, and highly active community of users.

Comments are closed on this article!

CATEGORIES

ALGORITHMS (/CS/CATEGORY/ALGORITHMS)

ARTIFICIAL INTELLIGENCE (/CS/CATEGORY/AI)

CORE CONCEPTS (/CS/CATEGORY/CORE-CONCEPTS)
DATA STRUCTURES (/CS/CATEGORY/DATA-STRUCTURES)
GRAPH THEORY (/CS/CATEGORY/GRAPH-THEORY)
LATEX (/CS/CATEGORY/LATEX)
NETWORKING (/CS/CATEGORY/NETWORKING)
SECURITY (/CS/CATEGORY/SECURITY)

(/cs/) (https://www.baeldung.com/cs/)

(/bael-search)

SERIES

ABOUT

ABOUT BAELDUNG (HTTPS://WWW.BAELDUNG.COM/ABOUT)
THE FULL ARCHIVE (/CS/FULL_ARCHIVE)
EDITORS (HTTPS://WWW.BAELDUNG.COM/EDITORS)

TERMS OF SERVICE (HTTPS://WWW.BAELDUNG.COM/TERMS-OF-SERVICE)
PRIVACY POLICY (HTTPS://WWW.BAELDUNG.COM/PRIVACY-POLICY)
COMPANY INFO (HTTPS://WWW.BAELDUNG.COM/BAELDUNG-COMPANY-INFO)
CONTACT (/CONTACT)